Code availability is not enough: schemas for documenting and extending data analysis

H. Sherry Zhang[1], Roger D. Peng[1]

ARTICLE HISTORY
Compiled October 11, 2026

[1] Department of Statistics and Data Science, University of Texas at Austin, Texas, United States

ABSTRACT
A data analysis involves a sequence of decisions, from data cleaning to model specification, each of which can affect the reported result. Reproducing that result is not the same as understanding the sequence of decisions that produced it and the reasons for each: the code mixes those decisions with implementation details, while the manuscript states the major decisions and their rationale but scatters them across its sections and often leaves out the minor ones. A reader may then ask how the result would change under a different decision: the question itself is easy to pose, but implementing it is often non-trivial in practice. In this work, we propose a schema that represents a data analysis as a collection of decisions in natural language, from which large language models (LLMs) can generate and validate analysis scripts directly. This workflow can also expand a single analysis into a full multiverse of alternatives, specified in an alternative file, ready for comparison. We demonstrate the proposed workflow in two applications. In the first, we construct a schema for a published regression analysis of heaped smoking data and show that it reproduces the reported estimates, and that a reviewer-suggested alternative can be specified through the alternative file and turned into code automatically. In the second, where no schema is provided, we show how the workflow serves instead as a reproducibility check, applied to the many-analysts football study. As an intermediate representation between the manuscript and code scripts, our schema gives a self-contained account of an analysis and makes it extensible from a reader's perspective.

KEYWORDS
Reproducibility; Large Language Models (LLMs); Data analysis decisions; Many-analysts study; Statistical workflow

CONTACT: H. Sherry Zhang. Email: hsherryzhang@utexas.edu.

1. Introduction

A data analysis is the product of a chain of decisions, from how the data are cleaned to how the model is specified. Those decisions are what a reader needs in order to comprehend an analysis, judge whether it is reasonable, and extend it by asking how the result would change under different choices. Standard practice in reproducible research (Peng, 2011; Wrobel et al., 2024) asks for the code and the data to be made available, but does not extend to the decisions behind them.

This gap arises because code and the decisions it implements operate at different levels: code specifies precisely what a computer should do, while a decision has a contextual interpretation that the code itself does not capture. Consider a line such as `dplyr::filter(df, x > 0)`, which has a precise meaning as code: keep rows where `x` is positive. Which decision this line encodes is not something the code can answer: it could represent restricting the analysis to a subpopulation for which $x > 0$ is substantively meaningful, or excluding observations flagged as invalid by an encoding convention where $x \leq 0$ denotes missingness, and nothing in the `filter()` syntax indicates which is intended. Nor does code generally carry the rationale behind a decision, unless the author has left detailed comments. A better place to state the decisions is the manuscript, where they are described along with their rationale. But this information is scattered across the manuscript rather than collected in one place, with preprocessing and modeling decisions appearing in different sections. Authors may also simply leave out decisions they consider minor, on the assumption that the code provides all the detail a reader would need. Reconstructing the full sequence, therefore, means searching across the paper and the code for decisions and their rationale before a reader can begin to comprehend the analysis.

Literate programming tools such as Sweave (Leisch, 2002), R Markdown (Xie et al., 2018), Jupyter notebooks (Kluyver et al., 2016), and Quarto (Allaire et al., 2026) integrate narrative and code within a single document, yet a published notebook does not resolve this problem. Substantive decisions, such as the exclusion of observations or the choice of covariates, remain interleaved with routine data manipulation, such as the renaming of columns, and a reader needs to examine the code to distinguish one from the other. Moreover, the narrative exists to explain the code, and the format places no requirement on what it must cover. An author narrates the steps they consider worth explaining, not the complete set of decisions needed to obtain the result. Ideally, a reader would understand an analysis by following its decisions step by step to reach the result, and would consult the code only when the implementation of a particular decision is of interest. What this requires is a self-contained record of the decisions and their rationale. With such a record, a reader can assess each decision that leads to the result and raise alternatives to investigate how the result changes. Once the chain of decisions has been specified, its implementation in code reduces largely to a translation task, which large language models (LLMs) now perform with reasonable reliability (Chen et al., 2021; Huang et al., 2022; Jiang et al., 2026; Lai et al., 2023; Shah et al., 2026).

In this paper, we present a schema that records a data analysis as a sequence of decisions, each stated with the objective it addresses and the rationale for choosing it, together with the variables it consumes and produces. We also demonstrate how this schema, coupled with LLMs, can be used to generate code scripts from natural-language descriptions in manuscripts and to explore alternative decisions. We present two applications. The first takes the perspective of an author supplying a schema alongside a published manuscript, using the GAITD regression analysis of heaped smoking data in Yee et al. (2025). A reader can then specify an alternative file, coupled with the original schema, to compare model performance under a different inflation-alteration specification. The second, the many-analysts football study, considers the opposite situation, where no schema is supplied and the workflow serves instead as a reproducibility test. It extracts a schema from a simplified version of each team's report, generates and validates the corresponding code, and compares the result against that team's published estimate.

The paper is organized as follows: In Section 2, we present our methodology, including the schema representation for describing a data analysis, the specification of the alternative file, and the workflow for generating and validating code from the schema with LLMs. In Section 3, we construct a schema for a published GAITD regression analysis and show how it allows a reader to evaluate an alternative specification directly. In Section 4, we apply the workflow as a reproducibility test to the many-analysts football study, recovering the reported results for most teams and discussing the failed cases. Finally, in Section 5, we discuss the implications of our work and potential future directions.

2. Methodology

2.1. *A schema for describing data analysis*

We represent a data analysis, $\mathcal{S} = \{A_1, A_2, \cdots, A_n\}$, as a sequence of n analysis decisions A_i, applied to a dataset $\mathcal{D}$, where each step, A_i, is defined as a tuple:

$$A_i = \langle f_i, \pi_i, \rho_i, \mathcal{I}_i, \mathcal{O}_i, \omega_i \rangle. \tag{1}$$

$f_i \in \mathcal{F}$ is the *objective* of a step, identifying the problem the *decision*, $\pi_i \in \Pi$, is trying to solve. ρ_i is the *rationale* provided for choosing the decision π_i. All three fields are written in short natural language sentences, and together they form a self-contained account of each decision in the analysis. As an example, Listing 1 presents the schema that describes the steps for calculating the Human Development Index (HDI) (United Nations Development Programme, 2025):

1) Scale all four variables (life expectancy, expected years of schooling, mean years of schooling, and GNI per capita) to the [0, 1] range.
2) Combine the two education variables (expected years of schooling and mean years

of schooling) into a single education variable.
3) Combine the three dimensions (health, education, and income) into a single index using a geometric mean.

Listing 1 An example schema for calculating Human Development Index (HDI)

```yaml
meta:
  type: schema
nodes:
- id: step-scaling
  objective: how to handle variables in different scales?
  decision: apply min-max scaling to each variable
  rationale: to convert the variables to the same scale for future
    construction of a single index
- id: step-education
  objective: how to combine the two education-related variables?
  decision: take the average
  rationale: the arithmetic mean allows perfect substitutability
    between the two variables
- id: step-combine
  objective: how to combine the three dimensions into a single index?
  decision: use the geometric mean
  rationale: the geometric mean penalizes uneven development
    among the three core dimensions
```

In the file, the `meta` field at the top marks the document as a schema (`type: schema`), and `nodes` below it records the analysis as a list of steps. Each step carries an `id` that names it, together with the `objective`, `decision`, and `rationale` fields holding f_i, π_i, and ρ_i. For instance, the objective of step 3 is to combine the three dimensions to create a single index. The geometric mean (the decision) is chosen over a simple arithmetic mean (an alternative, introduced in Section 2.2) because it penalizes uneven development among the three dimensions (the rationale).

Generating code from the schema also requires each step to be tied to concrete variables in the data. Where a dataset $\mathcal{D}$ is available, $\mathcal{I}_i$ and $\mathcal{O}_i$ record that mapping as part of the schema, so that it is fixed in advance rather than inferred at the point of code generation. Because the schema specifies an analysis before its code exists, $\mathcal{O}_i$ may name a variable that is not yet in the dataset. In contrast, $\mathcal{I}_i$ must already be in scope, comprising variables in the original dataset together with the outputs of all previous steps:

$$\mathcal{I}_i \subseteq \left(\mathcal{D} \cup \bigcup_{j<i} \mathcal{O}_j \right), \quad \forall i \in \{2, \ldots, n\}.$$

A schema can be written by hand, or drafted by an LLM from an existing description of

the analysis, such as the methodology section of a manuscript. In the latter case, $\mathcal{I}_i$ and $\mathcal{O}_i$ are inferred by the model whenever a dataset is provided, and each step carries a categorical confidence score, ω_i (low, medium, high), defined as the model's confidence that the recorded (f_i, π_i, ρ_i) correctly reflects the source text.

2.2. *Describing alternatives*

Recording a step's decision π_i does not mean it is the only one that could satisfy its objective f_i: a reader may want to revisit the decision made for some of these steps and consider alternatives that address the same objective, the premise of multiverse analysis (Simonsohn et al., 2020; Steegen et al., 2016). Formally, a step in the schema can have a candidate set of alternatives: step i may admit k_i decisions that all satisfy its objective f_i, $\{A_i^1, \ldots, A_i^{k_i}\}$, of which the decision π_i recorded in the schema is just one member, A_i^1. An alternative file specifies the remaining candidates, $A_i^2, \ldots, A_i^{k_i}$, for a subset of steps $C \subseteq \{1, \ldots, n\}$.

The overall structure of an alternative file is shown in Listing 2. The `meta` field at the top identifies the file as an alternative specification (`type: alternatives`), and the file-level `branch` field, either `single` or `multi`, determines how the candidates at each step are combined. The `nodes` field then holds one entry for every step in C. Each entry uses `overrides` to name the step it applies to in the original schema, then lists that step's alternative decisions and their rationale.

Listing 2 The skeleton of an alternative YAML file, which consists of a `branch` mode and a list of covered steps.

```yaml
meta:
  type: alternatives
  branch: single | multi
nodes:
  - overrides: <step id>
    alternatives:
      - id: <alternative id>
        decision: <...>
        rationale: <...>
      - id: <alternative id>
        ...
  - overrides: <step id>
    alternatives:
      ...
  ...
```

Under `branch: single`, every step is restricted to a single candidate decision. These alternatives, together with the rest of the original schema, form one new analysis. This is useful when an analyst already has a specific alternative analysis in mind and wants to evaluate it as a whole. Listing 3 fills in the skeleton to construct an alternative HDI computation, using a harmonic mean to combine the two education variables in Step 2

and an arithmetic mean to combine the three dimensions in Step 3.

Listing 3 A sample alternative file that changes the decisions for Step 2 and Step 3 in calculating Human Development Index (HDI).

```yaml
meta:
  type: alternatives
  branch: single
nodes:
  - overrides: step-education
    alternatives:
      - id: harmonic-mean
        decision: take the harmonic mean of the two education variables
        rationale: penalizes disparity between the two schooling measures
          rather than allowing one to compensate for the other
  - overrides: step-combine
    alternatives:
      - id: arith-mean
        decision: use an arithmetic mean
        rationale: a simple average of the three dimensions
```

Sometimes a reader instead wants to test a range of alternatives for one or more steps independently. Under `branch: multi`, each covered step may take any of its alternatives or keep its original decision, and every such combination across steps becomes a separate analysis. In Listing 4, one alternative is specified for the education step and two for the combination step. Together with the original schema, this produces $(1+1) \times (2+1) = 6$ analyses. More generally, with k_i candidates at each step $i \in C$, crossing all candidates yields $\prod_{i \in C} k_i$ analyses.

Listing 4 A sample alternative file that explores one alternative decision for Step 2 and two for Step 3 in calculating Human Development Index (HDI). Combining these alternatives with the original schema produces a total of (1+1) × (2+1) = 6 analyses.

```yaml
meta:
  type: alternatives
  branch: multi
nodes:
  - overrides: step-education
    alternatives:
      - id: harmonic-mean
        decision: take the harmonic mean of the two education variables
        rationale: penalizes disparity between the two schooling measures
          rather than allowing one to compensate for the other
  - overrides: step-combine
    alternatives:
      - id: arith-mean
        decision: use an arithmetic mean
        rationale: a simple average of the three dimensions
      - id: min-agg
```

```
    decision: use the minimum of the three dimensions
    rationale: a "weakest-link" aggregation that penalizes any single
      lagging dimension more strongly than an average would
```

Existing multiverse tooling, such as the `multiverse` package in R (Sarma et al., 2023), expresses each branch in code, while our alternatives are instead specified at the level of decisions, in natural language, from which modern LLMs can generate the corresponding code. Keeping them in a separate file also changes who can pose the alternative question: a multiverse is conventionally built by the original analyst exploring their own garden of forking paths, whereas an alternative file can be written by a reader after reading the published manuscript, against a schema the author has already supplied, to investigate questions that interest them.

2.3. *Workflow*

The reliability of the generated code depends primarily on the quality of the schema. A schema that faithfully records the decisions of an analysis, in the form set out in Section 2.1, leaves little to be interpreted at the code generation stage, since the objective and the decision at each step are stated explicitly. The errors that remain are in the code implementation rather than the analysis: a missing `library()` call, or a misnamed function argument.

The validation stage is designed to resolve errors of this kind. The validation agent executes the script in a sandbox environment and, when execution fails, receives the script together with the error message and the line number at which it occurred, and returns a revised script. The revised script is then executed again, and the cycle continues until the script completes without error, a maximum number of attempts is reached, or the user terminates the process. Application to the many-analysts football study in Section 4 shows that this procedure resolves most of the syntax errors that arise in data wrangling and statistical modeling.

The same generation-validation procedure applies to multiple analyses. When a schema is augmented with an alternative file, the expansion described in Section 2.2 produces one schema for each combination of decisions, and each resulting schema is passed through the same generation and validation procedure independently.

Figure 1 gives an overview of the workflow, which provides two paths for using a schema. Passing a schema through the code generation and validation stages produces a script whose execution yields the estimates of the analysis, which can be compared with the estimates originally reported, serving as a reproducibility check. A schema can also be augmented with an alternative file, expanding it into a multiverse of parallel analyses, so that the author and subsequent readers can explore the reported result under different decisions.

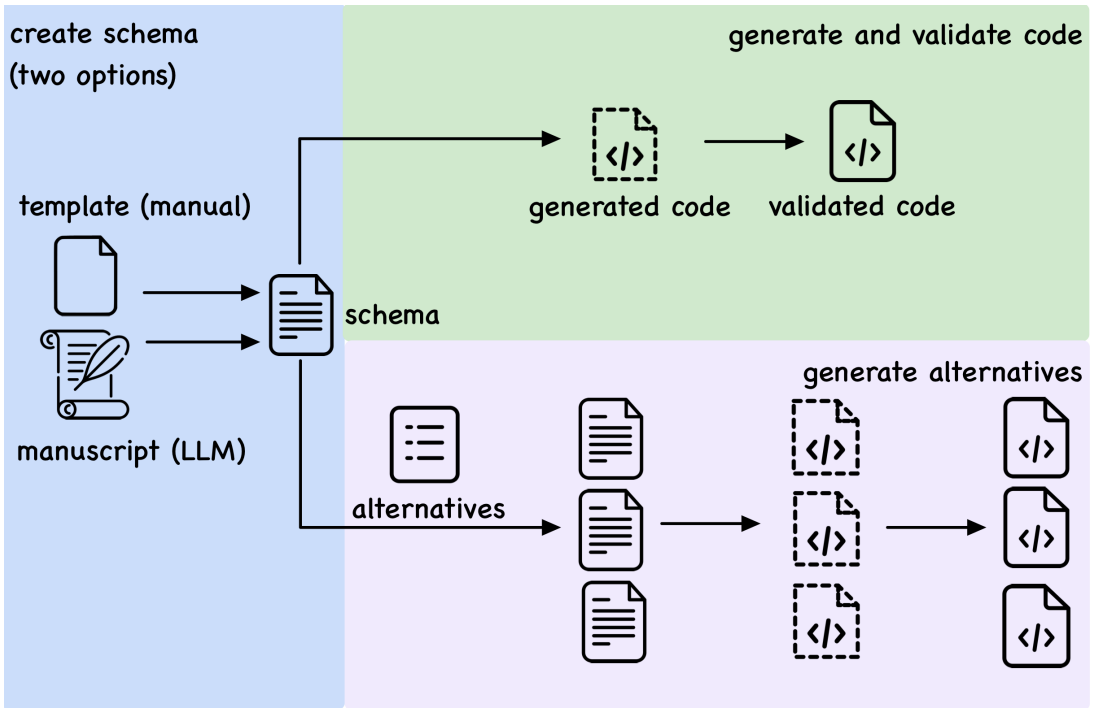


Figure 1. Overview of the schema-driven analysis workflow. Schemas can be authored manually or generated from natural language manuscripts by an LLM. Schemas are used to generate code, which is validated through iterative execution in a sandboxed environment and repaired when errors occur. Alternative specifications can expand a schema into a multiverse of parallel analyses.

3. Application: GAITD regression of heaped smoking data

In this section we demonstrate the workflow in the case where an author provides a faithful schema alongside a published manuscript, which a reader can then use to question and explore the analysis, following the lower path in Figure 1. We use Yee et al. (2025)'s GAITD regression analysis of heaped smoking data, published in the *Annals of Applied Statistics*, as the example. Yee et al. (2025) report four separate smoking-related responses, and for demonstration purposes, we build the schema for only the first response, smoking initiation age (P1, Section 5.1 of their paper). The same workflow extends to the other three cases.

The data come from the smoking questionnaire of the 2017–2020 cycle of the National Health and Nutrition Examination Survey (NHANES), available as `smqP` in the `VGAMdata` package (Yee & Gray, 2025). Each row is a survey respondent, recording their smoking history (initiation age `SIA`, cessation age `SCA`, and daily cigarette consumption, `TC30` for current smokers and `TCquit` for former smokers), their age, and the standard demographic covariates in smoking studies: gender, race, marital status, and education. Several of these variables are missing by design: `SCA` and `TCquit` exist only for former smokers, and education and marital status were not collected for respondents under 20; age is also top-coded at 80. The response, `SIA`, is a count whose distribution is heaped: certain ages are reported far more often than a classic count distribution would predict. GAITD (generally altered, inflated, truncated, and deflated) regression accommodates such heaping by modifying a parent count distribution, such as the Poisson or negative binomial, at specified values, which may receive additional probability (inflation), reduced probability (deflation), probabilities modeled separately from the parent (alteration), or no probability at all (truncation). Which ages to treat as special, and which operation to apply (inflation, deflation, alteration, or truncation) to each, are modeling decisions typically guided by an exploratory spikeplot of the empirical distribution of `SIA`.

The schema in Listing 5 comprises fourteen entries, each recording an *objective*, a *decision*, and a *rationale* in the structure introduced in Section 2.1. Consider `special_value_mid_risk`: the decision is to treat ages 12-14 and 19-21 as one inflated group with extra probability mass, since these ages show a moderate, secondary peak in the spikeplots with the high-risk group (ages 15-18) showing the most prominent peak. Recording the rationale along with the decision allows a reader to agree or disagree with the authors' choice, and to question and explore the analysis, as we do later in this section.

Listing 5 A schema for the GAITD regression analysis of smoking initiation age in Yee et al. (2025), constructed from the manuscript and intended to be provided alongside it as a self-contained account of the analysis.

```yaml
meta:
  type: schema
nodes:
- id: complete_case_filtering
```

```
  objective: handle missing data in selected variables
  decision: apply complete-case filtering to all columns except TC30, TCquit,
  ↪  and SCA which have NAs by design
  rationale: TC30, TCquit, and SCA are exempted from the completeness check
  ↪  since they are interlaced with NAs by design (e.g., SCA being NA means
  ↪  the person has not yet quit, a valid state rather than missingness);
  ↪  every other selected column is required to be non-missing because most
  ↪  smoking studies adjust for age, marital status, education and race, and
  ↪  this analysis follows suit.

- id: age_restriction
  objective: define the age range for analysis
  decision: restrict analysis to respondents aged less than 80 years (age <
  ↪  80)
  rationale: age was right-censored at 80

- id: data_quality_correction
  objective: handle logically impossible self-reported values
  decision: set both SCA and SIA to NA where SCA - SIA < 0
  rationale: quitting age before smoking initiation age is a logical
  ↪  impossibility and such values cannot be trusted

- id: impute_marital_young
  objective: assign a defensible marital status for young respondents,
  ↪  regardless of what was recorded
  decision: recode marital status to "Never married" for everyone age < 20
  rationale: this age group is effectively certain to have never been
  ↪  married, so the value is set by age rather than trusting the raw survey
  ↪  response

- id: impute_education_young
  objective: assign a defensible education category for young respondents,
  ↪  regardless of what was recorded
  decision: recode education to "< 9th grade" for age < 14; "9-11th grade"
  ↪  for 14 <= age <= 16; "High school" for 17 <= age <= 19
  rationale: for respondents still of school age, highest-grade-completed is
  ↪  essentially determined by age, so the value is set by age rather than
  ↪  trusting the raw survey response

- id: define_ever_smokers_sample
  objective: define the analytic sample of ever-smokers
  decision: include as valid observation if both conditions are met -- 1) SIA
  ↪  is not missing, and 2) either SCA and TCquit are both not missing, or
  ↪  SCA is missing and TC30 is not missing
  rationale: SCA is missing by design for anyone who has not yet quit. the
  ↪  two-branch rule instead admits ex-smokers through the SCA/TCquit branch
  ↪  and current smokers through the TC30 branch

- id: parent_distribution_choice
  objective: select appropriate probability distribution for count response
  ↪  variable
```

```
  decision: adopt negative binomial distribution as the parent distribution
  rationale: the SIA distributions iboth unimodal and right-skewed, so a NBD
  ↪  parent is reasonable; more generally, there is evidence of
  ↪  overdispersion relative to a Poisson distribution.

- id: special_value_mid_risk
  objective: specify how to model inflation at ages surrounding peak teenage
  ↪  smoking uptake
  decision: treat ages 12, 13, 14, 19, 20, 21 as a parametrically-inflated
  ↪  group, with the extra probability mass there sharing the same
  ↪  distributional shape (mean and dispersion) as the main population
  rationale: this mid-risk subgroup represents early and later smokers
  ↪  surrounding the high-risk age subgroup; these ages showed a moderate,
  ↪  secondary elevation in repeated training/test spikeplots of the data

- id: special_value_high_risk
  objective: specify how to model inflation at peak teenage smoking uptake
  ↪  ages
  decision: treat ages 15, 16, 17, 18 as a nonparametrically-inflated group,
  ↪  with each age given its own freely-estimated extra probability
  rationale: this high-risk subgroup corresponds to frenetic teenage smoking
  ↪  uptake, the age range where the largest number of regular smokers
  ↪  start; these ages showed the largest, most persistent spikes across
  ↪  repeated training/test spikeplots of the data.

- id: alteration_operator_decision
  objective: determine whether to apply alteration operators
  decision: do not use alteration operators (A.mix and A.mlm set to NULL)
  rationale: not needed for this model specification

- id: deflation_operator_decision
  objective: determine whether to apply deflation operators
  decision: do not use deflation operators (D.mix and D.mlm set to NULL)
  rationale: not needed for this model specification

- id: truncation_decision
  objective: determine whether to specify a truncated region
  decision: do not use truncation (no truncated region specified for SIA)
  rationale: truncation not appropriate for this response variable

- id: covariate_selection
  objective: select covariates for the linear predictor of parent
  ↪  distribution mean
  decision: include race2, gender, marital, and educ as covariates
  rationale: most smoking studies adjust for age, marital status, education
  ↪  and race

- id: fit_final_model
  objective: fit the final GAITD regression model
  decision: fit a single vglm that puts together the pieces above --
  ↪  response, parent distribution, special-value structure (inflation,
  ↪  alteration, deflation, and truncation), and covariates. The covariates
  ↪  only shift the mean of the negative binomial; the dispersion and the
  ↪  mid-/high-risk inflation probabilities are held fixed across the sample
```

```
  rationale: only the mean is modeled with covariates; the dispersion and
  ↪  inflation probabilities are kept intercept-only, since extending
  ↪  covariate effects to them would substantially increase the number of
  ↪  parameters to estimate and make the results harder to interpret
```

Assembling the schema does not require additional analysis from the authors, only the collection of decisions and rationale already present in the manuscript and supplementary material.

As a check that this schema is accurate, generating and validating code from it produces the R script in Listing 6, which reproduces the coefficients reported in Table 2 of Yee et al. (2025). Since the code was generated without access to the manuscript or the authors' script, the match shows that the schema records everything needed to reproduce the analysis. A schema of this kind could therefore accompany a manuscript as reproducibility material in its own right, alongside code and data, and a reader can rely on it as an accurate account of the analysis it documents.

Listing 6 The R script generated and validated from the schema in Listing 5, annotated with the `id` of the schema step each block implements.

```r
library(VGAMdata)
library(VGAM)
library(dplyr)

data("smqP", package = "VGAMdata")

cols <- c('TC30', 'TCquit', 'SCA', 'educ', 'marital',
          'SIA', 'SEQN', 'age', 'gender', 'race2',
          "passiveSmoke_job", "passiveSmoke_rest",
          "passiveSmoke_bar", "passiveSmoke_car",
          "passiveSmoke_home", "passiveSmoke_other",
          "passiveSmoke_ecigarettes")
df <- smqP |> select(all_of(cols))

# Step 1: complete_case_filtering
# Apply complete-case filtering to all columns except TC30, TCquit, and SCA
cols_require_complete <- setdiff(cols, c("TC30", "TCquit", "SCA"))
df_complete <- df |>
  filter(complete.cases(pick(all_of(cols_require_complete))))

# Step 2: age_restriction
# Restrict analysis to respondents aged less than 80 years
df_age_restricted <- df_complete |> filter(age < 80)

# Step 3: data_quality_correction
# Set both SCA and SIA to NA where SCA - SIA < 0
df_age_restricted <- df_age_restricted |>
  mutate(
    SCA_corrected = if_else(!is.na(SCA) & !is.na(SIA) & (SCA - SIA < 0),
      ↪  NA_integer_, SCA),
```

```
    SIA_corrected = if_else(!is.na(SCA) & !is.na(SIA) & (SCA - SIA < 0),
      ↪  NA_integer_, SIA)
  )

# Step 4: impute_marital_young
# Recode marital status to "Never married" for everyone age < 20
df_age_restricted <- df_age_restricted |>
  mutate(
    marital_imputed = if_else(age < 20, "Never married",
      ↪  as.character(marital))
  )

# Step 5: impute_education_young
# Recode education based on age for young respondents
df_age_restricted <- df_age_restricted |>
  mutate(
    educ_imputed = case_when(
      age < 14 ~ "< 9th grade",
      age >= 14 & age <= 16 ~ "9-11th grade",
      age >= 17 & age <= 19 ~ "High school",
      TRUE ~ as.character(educ)
    )
  )

# Step 6: define_ever_smokers_sample
# Include if: SIA not missing AND (both SCA and TCquit not missing OR SCA
  ↪  missing and TC30 not missing)
df_ever_smokers <- df_age_restricted |>
  filter(
    !is.na(SIA_corrected) &
    ((!is.na(SCA_corrected) & !is.na(TCquit)) | (is.na(SCA_corrected) &
  ↪  !is.na(TC30)))
  )

df_ever_smokers <- droplevels(df_ever_smokers)

# Step 7: parent_distribution_choice
# Adopt negative binomial distribution as the parent distribution
parent_distribution <- negbinomial

# Step 8: special_value_mid_risk
# Ages 12, 13, 14, 19, 20, 21 as parametrically-inflated group (i.mix)
i_mix_spec <- c(12, 13, 14, 19, 20, 21)

# Step 9: special_value_high_risk
# Ages 15, 16, 17, 18 as nonparametrically-inflated group (i.mlm)
i_mlm_spec <- c(15, 16, 17, 18)

# Step 10: alteration_operator_decision
# Do not use alteration operators
a_mix_spec <- NULL
```

```
a_mlm_spec <- NULL

# Step 11: deflation_operator_decision
# Do not use deflation operators
d_mix_spec <- NULL
d_mlm_spec <- NULL

# Step 12: truncation_decision
# Do not use truncation
truncation_spec <- NULL

# Step 13: covariate_selection
# Include race2, gender, marital, and educ as covariates
covariate_formula <- SIA_corrected ~ race2 + gender + marital_imputed +
  ↪  educ_imputed

# Step 14: fit_final_model
# Fit a single vglm with response, parent distribution, special-value
  ↪  structure, and covariates
fitted_model <- vglm(
  covariate_formula,
  family = gaitdnbinomial(
    i.mix = i_mix_spec, i.mlm = i_mlm_spec,
    a.mix = a_mix_spec, a.mlm = a_mlm_spec,
    d.mix = d_mix_spec, d.mlm = d_mlm_spec,
    truncate = truncation_spec
  ),
  data = df_ever_smokers
)
```

Yee et al. (2025)'s supplementary material reports an investigation from a reviewer, who suggested splitting the mid-risk group into two, keeping ages 12-14 as inflated but treating ages 19-21 as altered instead, since that older subgroup shows a qualitatively different, declining pattern of uptake. Listing 7 encodes the reviewer's suggestion as an alternative file, overriding three steps at once: `special_value_mid_risk` narrows the inflated group to ages 12-14, `alteration_operator_decision` turns on an alteration group for ages 19-21, and `covariate_selection` drops to an intercept-only model, matching how the authors themselves computed AIC for this comparison; `branch: single` ties the three overrides into one refit, rather than crossing them into separate branches. Generating and validating code from this alternative file reproduces the AIC of 19,340.4 that Yee et al. (2025) reports. Here the question was answered because the authors chose to report the refit in their supplement; with the schema available, any reader who raises such a question can answer it directly.

Listing 7 An alternative YAML file specifying a joint change to the mid-risk inflation, alteration, and covariate steps in the GAITD schema, treating ages 19-21 as an alteration group rather than pooling them into the mid-risk inflated group.

```yaml
meta:
  type: alternatives
  branch: single
nodes:
  - overrides: special_value_mid_risk
    alternatives:
      - id: ap1921-special-value
        decision: "treat ages 12, 13, 14 only as a parametrically-inflated
  ↪  group, with the extra probability mass there sharing the same
  ↪  distributional shape (mean and dispersion) as the main population; ages
  ↪  19, 20, 21 are no longer pooled into this group (see the alteration
  ↪  operator decision instead)"
        rationale: "ages 19-21 mark less frenetic, declining smoking uptake
  ↪  as respondents age out of the high-risk window, a qualitatively
  ↪  different pattern from the early uptake at ages 12-14, so they are
  ↪  moved out of this pooled inflated group"
  - overrides: alteration_operator_decision
    alternatives:
      - id: ap1921-alteration
        decision: "use ages 19, 20, 21 as an alteration group (A.mix) instead
  ↪  of leaving alteration operators unused"
        rationale: "ages 19-21 mark less frenetic, declining smoking uptake
  ↪  as respondents age out of the high-risk window, a qualitatively
  ↪  different pattern from the early uptake at ages 12-14, so they are
  ↪  represented as an alteration rather than folded into the same inflated
  ↪  group"
  - overrides: covariate_selection
    alternatives:
      - id: ap1921-covariates
        decision: "do not include any covariates (intercept-only model)"
        rationale: "this comparison is about which special-value structure
  ↪  fits better, so covariates are left out to compare the two structures
  ↪  on equal footing, the same way the special-value structure was first
  ↪  established on an intercept-only model before covariates were added"
```

4. Application: many-analysts football study

Section 3 assumed the authors supply a schema alongside the manuscript, but a schema need not come from the authors. Since it records decisions in natural language, the schema can be extracted from the manuscript itself by an LLM for readers to understand and explore the analysis. In this section, we demonstrate the workflow in the opposite situation, where no schema is supplied and the workflow serves as a reproducibility test using the many-analysts football study (Silberzahn et al., 2018). We choose this study because its many-analysts design provides replicates of the same analysis: independent teams analyzed the same data to answer the same question, each reporting its methodology and estimate.

Silberzahn et al. (2018) invited 29 research groups to conduct the same data analysis,

answering whether referees are more likely to give red cards to dark-skin-toned players than to light-skin-toned players. The provided dataset contains two raters' skin color ratings of players' pictures, the aggregated number of red cards received by a player from a referee across all matches in that player-referee pair, and other relevant player, referee, and game characteristics: player birthday, height, weight, player position, league country, number of victories, ties, and defeats.

Each team was free to choose their own analysis approach; we take Team 5 as an illustration. They disaggregated the player-referee records into player–game observations, giving a binary response for whether a player received a red card, and removed incomplete cases by list-wise deletion. They then fitted a binomial generalized linear mixed model with a logit link, taking the average of the two raters' skin color ratings as the covariate of interest, with random intercepts for players and referees and a random intercept and slope for the referee's country of origin.

Team 5's analysis is only one of the approaches taken: other teams made different preprocessing decisions, assumed different distributions (linear, logistic, or Poisson), specified different fixed and random effects, or worked in a Bayesian framework. Each team was required to document its analysis in a report with a publishable methodology section. All of these are available on the Open Science Framework.

These reports are not directly usable as input since the many-analysts study also takes in a second research question and the construction of confidence intervals. Some teams write their final analysis as a set of edits to the initial one rather than as a self-contained methodology section. We therefore pre-extract a simplified methodology text for each team and use it in place of the report.

Each team's text is then passed through the workflow of Section 2 to extract the schema and then generate and validate the corresponding code with Claude Opus 4.5. The Appendix follows Team 5 through the pipeline to show the input text, the schema, and the generated script.

To accelerate the evaluation of each script, we manually replace the standard modeling functions with faster counterparts after the scripts are generated but before validation: `lm()`and `glm()` with `fixest::feols()` and `fixest::feglm()` (Bergé et al., 2026), and `lme4::glmer()` (Bates et al., 2015) with `glmmTMB::glmmTMB()` (Brooks et al., 2017). These fit the same models by faster means, using multi-threaded C++ and Template Model Builder (TMB) respectively, so the substitution changes how the coefficients are computed, not the coefficients themselves.

Of the 29 teams, the analyses of four teams (Teams 7, 12, 17, and 18) are too computationally expensive to run inside an iterative validate-and-repair loop, regardless of whether the schema or generated code is correct: Team 7 specifies a Dirichlet process profile regression, Team 12 a zero-inflated Poisson regression, and Teams 17 and 18 MCMC-based hierarchical models. Hence we exclude these four teams from the evaluation, leaving 25 teams whose analyses we attempt to reproduce. Across the 25 teams

we reproduce, all team scripts were successfully executed after the validation phase. Notably, two scripts ran successfully without any validation, whereas the vast majority reached completion after two or three iterations. The most frequent correction involved resolving broken and overflowing syntax in comments.

Figure 2 shows the comparison of the estimated effect sizes, in odds ratio, from our schema workflow (reproduced) and those reported in Silberzahn et al. (2018) (reported). The odds ratio is well recovered for most teams, though six show a noticeably larger gap: Teams 1, 9, 15, 21, 23, and 27.

For Teams 1 and 23 the discrepancy originates in our code generation. When reshaping the data from the player-referee level to the game level, the schema correctly specifies this step, but the generated code fails to execute it as intended, and the resulting error passes through validation silently rather than triggering a repair.

For the remaining four the reported result could not be regenerated from the report, though one of them resists diagnosis: for Team 21, examining both the extracted schema and the generated code turned up nothing that accounts for the gap. The other three reveal reproducibility issues in the reports themselves. Team 9's text names "referee country" as the covariate while the full report shows "player country" is used instead. For Team 27, the authors write that "a variable for player position was included, as well as a variable for the league of the player", which read literally adds only main effects but might equally be shorthand for interactions, consistent with how the other covariates entered their initial model. Team 15 is subtler: their log-linear model includes a skin-tone by league interaction, so the coefficient of interest is defined relative to a parameterization the report does not state, with SPSS's effect coding averaging it across leagues and R's default dummy coding fixing it at a reference league. Setting the `contrasts` argument in R recovers the reported value exactly, so the discrepancy is numerical rather than statistical, although identifying it required manual investigation.

4.1. *Exploring alternatives*

Team 5 arrives at its final model by comparing three random effect structures, a set of alternative decisions within a single modeling step of the kind described in Section 2.2. Write y_{ijm} for whether player i received a red card from referee j in game m, x_i for player i's skin color rating averaged over the two raters and rescaled to $[0, 1]$, and $c(j)$ for the country of origin of referee j. In each model $y_{ijm} \sim \text{Bernoulli}(p_{ijm})$, and the three differ only in their random effects:

$$
\begin{aligned}
\textbf{GM1:} \quad & \text{logit}(p_{ijm}) = \beta_0 + \beta_1 x_i + u_i + v_j, \\
\textbf{GM2:} \quad & \text{logit}(p_{ijm}) = \beta_0 + \beta_1 x_i + u_i + v_{0j} + v_{1j}\, x_i, \\
\textbf{GM3:} \quad & \text{logit}(p_{ijm}) = \beta_0 + \beta_1 x_i + u_i + v_j + w_{0,c(j)} + w_{1,c(j)}\, x_i,
\end{aligned}
$$

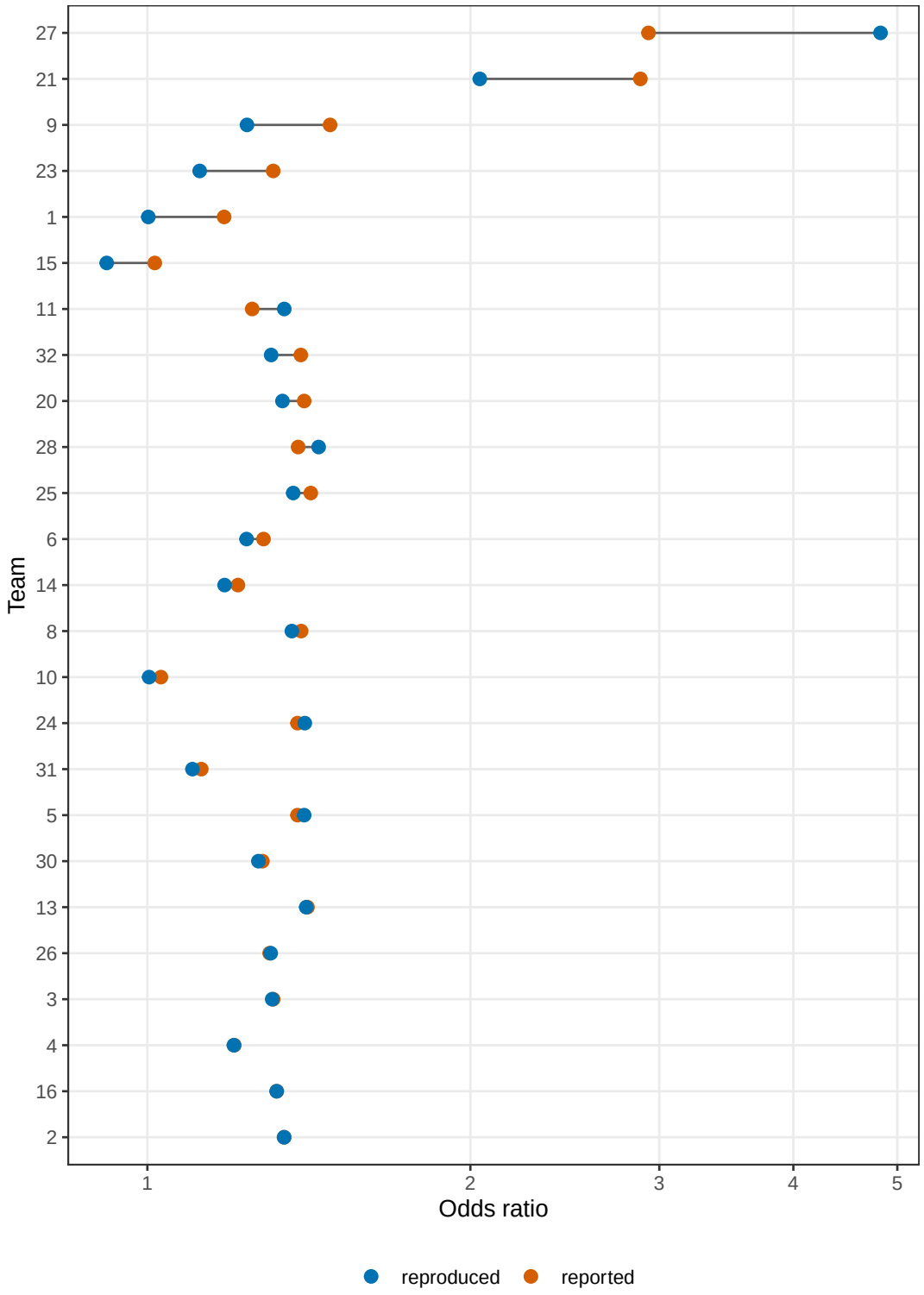


Figure 2. Comparison of the odds ratio of the effect of skin color on receiving red cards in football games from our schema-based reproduction (reproduced) vs. the effect size reported in Silberzahn et al. (2018) (reported). Teams are ordered by the absolute difference between the reproduced and reported odds ratio. The reproduction recovers the reported odds ratio closely for most teams; the largest deviations occur for Team 1, 9, 15, 21, 23, and 27.

with $u_i \sim N(0, \sigma^2_{\text{player}})$ throughout, $v_j \sim N(0, \sigma^2_{\text{ref}})$ in GM1 and GM3, $(v_{0j}, v_{1j})^\top \sim N(\mathbf{0}, \mathbf{\Sigma}_{\text{ref}})$ in GM2, and $(w_{0c}, w_{1c})^\top \sim N(\mathbf{0}, \mathbf{\Sigma}_{\text{country}})$ in GM3. Model GM1 lets the baseline log-odds vary across players and referees, GM2 additionally lets the effect of skin color vary across referees, and GM3, the model Team 5 report, instead lets both the baseline and that effect vary across referees' countries of origin.

All three alternatives are specified in a single YAML file, following the format described in Section 2.2 and shown in Listing 8. Merging it with Team 5's schema produces one schema per alternative and yield three validated scripts. Both estimated random and fixed effects in each model matches with Team 5's reported values.

Listing 8 Example of a YAML file specifying alternatives for the random effects structure in the many-analysts analysis of Team 5.

```yaml
meta:
  type: alternatives
  branch: multi
nodes:
- overrides: specify_random_effects_structure
  alternatives:
  - id: gm1
    decision: a random effect of player and referee
    rationale: ''
  - id: gm2
    decision: a random effect of player and referee, and a random effect of
   ↪  skin-tone across referees
    rationale: ''
  - id: gm3
    decision: a random effect of player, referee, and referees' countries of
   ↪  origin, and a random effect of skin tone across referees' countries of
   ↪  origin
    rationale: ''
```

5. Discussion

Existing reproducible practices focus on whether published results can be reproduced from the code and data, not on the chain of decisions that produced them. We propose a schema that records those decisions along with the rationale for choosing them in natural language as an intermediate representation between the manuscript and the code. A schema can be written by an author or extracted from the manuscript by an LLM. It can then be turned into an executable script, or expanded into a multiverse of alternative analyses. For a published GAITD regression, where the schema was assembled from the manuscript and its supplement, the resulting script reproduced the reported estimates and brought a reviewer's alternative specification within reach of any reader. For the many-analysts football study, where no schema was supplied, extraction and generation recovered most teams' reported effects and identified the few where the methodology

could have been described more precisely, inconsistencies that re-running the authors' own code would not reveal.

A data analysis question admits more than one defensible answer, and the schema records which one was taken. Arriving at those decisions is a matter of human judgement, whether the analyst writes the schema directly or it is extracted from a manuscript in which the decisions were already made. Once they are determined, generating the code is a comparatively mechanical translation, which current LLMs perform with reasonable reliability inside a generation-and-validation loop. Our approach therefore contrasts with fully automated systems that use multiple LLM agents to plan, generate, and execute analyses (Hu et al., 2024; Jansen et al., 2025; Sun et al., 2026), which aim to automate the decisions themselves. Donoho (2026) describes the risk in such systems as one of premature autonomy, where the model projects an intent onto the user rather than responding to what was asked. We do not attempt to automate the human judgement that goes into making those decisions, but instead provide a structured way to record them and a workflow to turn them into code.

Three sources of uncertainty enter this pipeline, and they behave differently as models improve. The first is implementation, where the schema states a decision unambiguously and the generated code fails to carry it out: the scripts for Teams 1 and 23 ran and returned estimates after reshaping the data incorrectly. The second is extraction, where the text states a decision clearly to a trained reader and the model fails to recover it. In Team 5's report, the phrase "a random effect of skin tone across referees' countries" is read by most statisticians as implying both a random intercept and a slope, while read literally it specifies only a slope, and the extraction took the literal reading rather than the conventional one. Both are epistemic (reducible) (Hüllermeier & Waegeman, 2021; Kendall & Gal, 2017) and we would expect them to diminish as models continue to improve, rather than persist as structural limitations of the workflow. The third is aleatoric (irreducible) and belongs to the source text rather than the pipeline: where a report is ambiguous or self-contradictory, as in Teams 9, 27, and 15, no model can recover an intent the text does not express. Team 15 differs in kind: its model is fully specified, and the discrepancy is numerical, arising because a reparameterization changes which quantity a coefficient denotes. It remains a reproducibility issue, since the reported number could be regenerated only by manually investigating the encoding. As extraction and generation improve, what remains is increasingly uncertainty of this third kind, which the schema is designed to make visible.

The validate-and-repair loop has two limitations. First, it requires each candidate script to be executed and diagnosed within an interactive timeframe, which precludes computationally intensive analyses. For this reason, the analyses of Teams 7, 12, 17, and 18 were excluded from the evaluation. Second, for privacy reasons the LLM is given the column names of the data but not their values, so any data format that the schema does not specify must be inferred through validation. The birth dates in the football data, from which player age is derived, illustrate this: the script for Team 21 initially parsed them as `%Y-%m-%d` and adopted the correct `%d.%m.%Y` only after a parsing failure,

whereas it takes Team 1 four attempts to find the correct formats. In both cases the format was recovered by trial and error and had these attempts failed, the pipeline would either have terminated or propagated the error silently into the analysis.

We conclude with several directions for future work. Executing scripts through a background job queue would accommodate analyses too expensive for an interactive loop without changing the rest of the workflow. The implementation reported here is an R package, but nothing about the problem is specific to R, and a general-purpose tool such as a command-line utility would support analyses written in any language. Our implementation records the schema as a single YAML file, which gives the decisions an explicit structure that machines can read. An alternative is to record each decision in its own Markdown file to trade some of that structure for flexibility in how a decision is described. Separate files would further allow decisions to be linked over time, showing how an analysis developed rather than only where it arrived, and with attribution to individual researchers such a history could support a broader research-management system.

6. Acknowledgments

The implementation of the schema is available on CRAN as an R package, `tines`. The source code for reproducing the work reported in this paper is available in the Supplementary Materials and can be found at: https://github.com/huizezhang-sherry/paper-schema.

All analyses were carried out in R (R Core Team, 2026), using the `tines`, `tidyverse` (Wickham et al., 2019), `VGAM` (Yee, 2025), `VGAMdata` (Yee & Gray, 2025), `lme4` (Bates et al., 2015), `glmmTMB` (Brooks et al., 2017; McGillycuddy et al., 2025), and `fixest` (Bergé et al., 2026) packages. The article is created using Quarto (Allaire et al., 2026).

References

Allaire, J. J., Teague, C., Scheidegger, C., Xie, Y., Dervieux, C., & Woodhull, G. (2026). *Quarto* (Version 1.9) [Computer software]. https://doi.org/10.5281/zenodo.5960048

Bates, D., Mächler, M., Bolker, B., & Walker, S. (2015). Fitting linear mixed-effects models using lme4. *Journal of Statistical Software*, *67*(1), 1–48. https://doi.org/10.18637/jss.v067.i01

Bergé, L. R., Butts, K., & McDermott, G. (2026). Fast and user-friendly econometrics estimations: The R package fixest. *arXiv Preprint arXiv:2601.21749*. https://doi.org/10.48550/arXiv.2601.21749

Brooks, M. E., Kristensen, K., van Benthem, K. J., Magnusson, A., Berg, C. W., Nielsen, A., Skaug, H. J., Maechler, M., & Bolker, B. M. (2017). glmmTMB balances speed and flexibility among packages for zero-inflated generalized linear mixed modeling. *The R Journal*, *9*(2), 378–400. https://doi.org/10.32614/RJ-2017-066

Chen, M., Tworek, J., Jun, H., et al. (2021). Evaluating large language models trained on code. *arXiv Preprint arXiv:2107.03374*. https://arxiv.org/abs/2107.03374

Donoho, D. (2026). Discussion of "LAMBDA: A large model based data agent". *Journal of the American Statistical Association*, *121*(553), 14–16. https://doi.org/10.1080/01621459.2025.2574396

Hu, S., Lu, C., & Clune, J. (2024, October). *Automated Design of Agentic Systems*. https://openreview.net/forum?id=t9U3LW7JVX

Huang, J., Wang, C., Zhang, J., Yan, C., Cui, H., Inala, J. P., Clement, C., & Duan, N. (2022). Execution-based Evaluation for Data Science Code Generation Models. In E. Dragut, Y. Li, L. Popa, S. Vucetic, & S. Srivastava (Eds.), *Proceedings of the Fourth Workshop on Data Science with Human-in-the-Loop (Language Advances)* (pp. 28–36). Association for Computational Linguistics. https://doi.org/10.18653/v1/2022.dash-1.5

Hüllermeier, E., & Waegeman, W. (2021). Aleatoric and epistemic uncertainty in machine learning: An introduction to concepts and methods. *Machine Learning*, *110*(3), 457–506.

Jansen, J. A., Manukyan, A., Khoury, N. A., & Akalin, A. (2025). Leveraging large language models for data analysis automation. *PLOS ONE*, *20*(2), e0317084. https://doi.org/10.1371/journal.pone.0317084

Jiang, J., Wang, F., Shen, J., Kim, S., & Kim, S. (2026). A survey on large language models for code generation. *ACM Transactions on Software Engineering and Methodology*, *35*(2), 1–72.

Kendall, A., & Gal, Y. (2017). What uncertainties do we need in bayesian deep learning for computer vision? *Advances in Neural Information Processing Systems 30 (NeurIPS 2017)*, 5574–5584.

Kluyver, T., Ragan-Kelley, B., Pérez, F., Granger, B., Bussonnier, M., Frederic, J., Kelley, K., Hamrick, J., Grout, J., Corlay, S., Ivanov, P., Avila, D., Abdalla, S., Willing, C., & team, J. development. (2016). Jupyter notebooks - a publishing format for reproducible computational workflows. In F. Loizides & B. Scmidt (Eds.), *Positioning and power in academic publishing: Players, agents and agendas* (pp. 87–90). IOS Press. https://eprints.soton.ac.uk/403913/

Lai, Y., Li, C., Wang, Y., Zhang, T., Zhong, R., Zettlemoyer, L., Yih, W.-T., Fried, D., Wang, S., & Yu, T. (2023). DS-1000: A Natural and Reliable Benchmark for Data Science Code Generation. *Proceedings of the 40th International Conference on Machine Learning*, 18319–18345. https://proceedings.mlr.press/v202/lai23b.html

Leisch, F. (2002). Sweave: Dynamic generation of statistical reports using literate data analysis. In W. Härdle & B. Rönz (Eds.), *Compstat 2002 — proceedings in computational statistics* (pp. 575–580). Physica-Verlag GmbH.

McGillycuddy, M., Warton, D. I., Popovic, G., & Bolker, B. M. (2025). Parsimoniously fitting large multivariate random effects in glmmTMB. *Journal of Statistical Software*, *112*(1), 1–19. https://doi.org/10.18637/jss.v112.i01

Peng, R. D. (2011). Reproducible Research in Computational Science. *Science*, *334*(6060), 1226–1227. https://doi.org/10.1126/science.1213847

R Core Team. (2026). *R: A language and environment for statistical computing*. R Foundation for Statistical Computing. https://doi.org/10.32614/R.manuals

Sarma, A., Kale, A., Moon, M. J., Taback, N., Chevalier, F., Hullman, J., & Kay, M. (2023). Multiverse: Multiplexing alternative data analyses in r notebooks. *Proceedings of the 2023 CHI Conference on Human Factors in Computing Systems*. https://doi.org/10.1145/3544548.3580726

Shah, S. M. H., Hopfgartner, F., & Bleier, A. (2026). *Automating Computational Reproducibility in Social Science: Comparing Prompt-Based and Agent-Based Approaches*. https://doi.org/10.1145/3774905.3795485

Silberzahn, R., Uhlmann, E. L., Martin, D. P., Anselmi, P., Aust, F., Awtrey, E., Bahník, Š., Bai, F., Bannard, C., Bonnier, E., Carlsson, R., Cheung, F., Christensen, G., Clay, R., Craig, M. A., Dalla Rosa, A., Dam, L., Evans, M. H., Flores Cervantes, I., ... Nosek, B. A. (2018). Many Analysts, One Data Set: Making Transparent How Variations in Analytic Choices Affect Results. *Advances in Methods and Practices in Psychological Science*, *1*(3), 337–356. https://doi.org/10.1177/2515245917747646

Simonsohn, U., Simmons, J. P., & Nelson, L. D. (2020). Specification curve analysis. *Nature Human Behaviour*, *4*(11), 1208–1214. https://doi.org/10.1038/s41562-020-0912-z

Steegen, S., Tuerlinckx, F., Gelman, A., & Vanpaemel, W. (2016). Increasing transparency through a multiverse analysis. *Perspectives on Psychological Science*, *11*(5), 702–712. https://doi.org/10.1177/1745691616658637

Sun, M., Han, R., Jiang, B., Qi, H., Sun, D., Yuan, Y., & Huang, J. (2026). LAMBDA: A Large Model Based Data Agent. *Journal of the American Statistical Association*, *121*, 1–13. https://doi.org/10.1080/01621459.2025.2510000

United Nations Development Programme. (2025). *Human development report 2025*. United Nations Development Programme. https://hdr.undp.org/

Wickham, H., Averick, M., Bryan, J., Chang, W., McGowan, L. D., François, R., Grolemund, G., Hayes, A., Henry, L., Hester, J., Kuhn, M., Pedersen, T. L., Miller, E., Bache, S. M., Müller, K., Ooms, J., Robinson, D., Seidel, D. P., Spinu, V., ... Yutani, H. (2019). Welcome to the tidyverse. *Journal of Open Source Software*, *4*(43), 1686. https://doi.org/10.21105/joss.01686

Wrobel, J., Hector, E. C., Crawford, L., McGowan, L. D., Silva, N. da, Goldsmith, J., Hicks, S., Kane, M., Lee, Y., Mayrink, V., Paciorek, C. J., Usher, T., & Wolfson, J. (2024). Partnering With Authors to Enhance Reproducibility at JASA. *Journal of the American Statistical Association*, *119*(546), 795–797. https://doi.org/10.1080/01621459.2024.2340557

Xie, Y., Allaire, J. J., & Grolemund, G. (2018). *R markdown: The definitive guide*. Chapman; Hall/CRC. https://bookdown.org/yihui/rmarkdown

Yee, T. W. (2025). *VGAM: Vector generalized linear and additive models*. https://CRAN.R-project.org/package=VGAM

Yee, T. W., Frigau, L., & Ma, C. (2025). Heaping and seeping, GAITD regression and doubly constrained reduced-rank vector generalized linear models in smoking studies. *The Annals of Applied Statistics*, *19*(4), 3045–3070.

Yee, T. W., & Gray, J. (2025). *VGAMdata: Data supporting the VGAM package*. https://CRAN.R-project.org/package=VGAMdata